OẲN TÙ TÌ - AI ARENA

Hướng dẫn sử dụng và vận hành giải đấu

Bản triển khai Docker | Tiếng Việt | 23/10/2026

Nền tảng tổ chức giải lập trình bot Python trên bàn cờ 9 × 9: ban tổ chức tạo giải, xác nhận đội, nhận bài, chia bảng, chấm tự động, theo dõi replay và công bố kết quả.

Địa chỉ trên máy hiện tại: http://localhost:3000/competition

Mã nguồn: https://github.com/nguyenlean17/oan-tu-ti-ai-arena

Tài liệu này mô tả phần đã triển khai. Giải có chữ DEMO gồm tài khoản và bot mẫu phục vụ kiểm thử, không phải sinh viên thật. Các lệnh minh họa dùng PowerShell trên Windows; file compose cũng chạy được trên Linux.

Đọc theo vai trò

Bạn là ai?	Đọc phần nào?
Người cài đặt	Khởi động, sơ đồ triển khai, sao lưu, đưa lên Internet
Ban tổ chức	Tạo giải, danh sách đội, cấu hình, khóa bài, công bố
Sinh viên	Tham gia đội, viết bot, nộp bài, xem kết quả
Người bảo trì	Ranh giới bảo mật, kiểm thử, xử lý sự cố

Phiên bản này ưu tiên một máy chủ điều hành, một ổ dữ liệu bền vững và các worker có thể nhân bản. Không cần Redis, Kafka hoặc Kubernetes. Vercel/Sites cũ là không gian luyện tập riêng; giải chính thức chạy trên bản Docker có worker dài hạn.

1. Khởi động trên máy của bạn

Cần Docker Desktop chạy Linux containers, Node.js 22.16 trở lên và bản sao repository. Gợi ý máy thử nghiệm: 4 luồng CPU, RAM 8 GB, SSD còn trống 10 GB. Đây là cấu hình khởi đầu để đo tải, không phải cam kết cho 4.000 người cùng lúc.

Trong thư mục dự án:

```
npm run competition:init
docker compose up -d --build --wait
docker compose ps
```

Lệnh đầu tạo secret ngẫu nhiên tại `.secrets/judge-token`, không ghi đè secret đã có. Không đưa thư mục này lên Git. Lần build đầu cần Internet để lấy image và runtime Python; khi chấm không tải thư viện của sinh viên.

Container chạy với UID 1000. Trên Linux, nếu tài khoản cài đặt có UID khác 1000, cấp chủ sở hữu tệp token bằng `sudo chown 1000:1000 .secrets/judge-token`. Khi chạy bước khởi tạo bằng root, tiện ích thực hiện việc này tự động.

Mở http://localhost:3000/competition. Đăng ký tài khoản của bạn tại `/register`; mật khẩu 12-128 ký tự. Sau đó chủ máy cấp quyền BTC cho đúng email:

```
docker compose exec app node docker/admin.mjs your@email.com
```

Tải lại trang, mở **Ban tổ chức**. Quyền này áp dụng ngay cho giải chính thức; không cần bước nhận quyền ở admin panel cũ của khu luyện tập.

Dữ liệu được giữ ở đâu?

Volume `oan-tu-ti-arena_arena-data` chứa `auth.sqlite`, `competition.sqlite`, `casual.sqlite`. Khởi động lại hoặc build lại không xóa dữ liệu. Bản Docker cũ được sao chép dữ liệu luyện tập sang SQLite mới và giữ lại bản gốc.

```
docker compose stop
docker compose up -d
```

Không dùng `docker compose down -v` với giải cần giữ. Các tài khoản kiểm thử cục bộ không phải tài khoản mặc định của sản phẩm. Thông tin tài khoản BTC demo, nếu đã chạy bộ kiểm thử, nằm riêng trong `.secrets/local-demo-admin.json`.

2. BTC: tạo giải và xác nhận đội

Mở **Ban tổ chức**, bấm **Tạo giải đấu mới** (hoặc chọn **Tất cả giải đấu**), đặt tên giải, nạp cấu hình mặc định rồi bấm **Tạo giải**. Cấu hình mặc định là 4 bảng, 2 suất đi tiếp/bảng, tối đa 200 lượt/ván.

Nhập danh sách từ Excel vào ô TSV, không có tiêu đề. Các cột được ngăn bằng phím Tab:

```
Tên đội    Hạt giống    Email    MSSV    Repo GitHub (tùy chọn)
```

Mỗi đội có tên và số hạt giống duy nhất trong giải. Nếu có nhiều thành viên, dùng thêm dòng cùng tên đội hoặc mục **Thêm thành viên vào đội**. BTC phải đối chiếu danh sách trường/lớp trước khi cấp mã.

Bấm **Nhập đội và cấp mã**, tải tệp mã ngay và gửi riêng từng mã cho đúng người. Mã có hạn 7 ngày, dùng một lần, ràng buộc email và MSSV. Hệ thống chỉ lưu hash nên không thể xem lại mã cũ. Cấp lại sẽ hủy mã cũ chưa dùng của MSSV đó.

Sinh viên đăng ký đúng email rồi đổi mã trong **Bài nộp của đội**. Một tài khoản chỉ thuộc một đội trong mỗi giải. Các thành viên cùng đội đều có quyền nộp; phiên bản mới nhất của cả đội là phiên bản dùng chung.

Chống nộp thay người khác

Trình duyệt không được tự chọn ID đội để ghi bài. Máy chủ lấy đội từ phiên đăng nhập và membership đã đổi mã. Tên hiển thị, email tự khai và tác giả commit không chứng minh danh tính. Mã mời là bằng chứng do BTC phát qua kênh riêng; nếu bị lộ phải thu hồi quyền và cấp lại sau khi xác minh.

Bấm **Thu hồi quyền** ở thành viên để chặn quyền nộp trước lúc khóa danh sách. Không gửi toàn bộ tệp mã cho cả lớp.

3. BTC: cấu hình và vận hành giải

Có thể nhập/xuất config JSON. Máy chủ kiểm tra các giá trị trước khi lưu.

Khóa	Ý nghĩa và giới hạn
groupCount	1-128 bảng, mỗi bảng 2-32 đội có bài hợp lệ
qualifiers	1-4 suất/bảng; tổng suất phải là lũy thừa của 2, tối thiểu 2
deadline	Thời điểm ISO UTC, ví dụ 2026-11-01T12:00:00.000Z
seed	Số nguyên dùng cho chia bảng và seed trận
maxPlies	20-200 lượt đi; hết lượt mà chưa thắng thì hòa
moveMs	100-5.000 ms cho mỗi nước đi
startupMs	1.000-15.000 ms để nhập bot và khởi tạo
memoryMB	64-256 MiB bộ nhớ linear WebAssembly mỗi bot
extraSets	0-2 cặp đấu phụ khi vòng loại trực tiếp hòa

1. **Bản nháp:** sửa cấu hình, thêm đội, cấp mã.
2. **Mở nhận bài:** nhận bài Python và kiểm tra tự động. Từ đây chỉ sửa được hạn nộp; luật và giới hạn tài nguyên đã cố định.
3. **Khóa bài & chia bảng:** chỉ thực hiện sau hạn. Nếu còn bài chờ kiểm tra, trạng thái chuyển sang đang chốt; đợi worker xong rồi bấm khóa lại.
4. Hệ thống chọn bài hợp lệ mới nhất trước hạn, cố định danh sách và tự chấm vòng bảng rồi vòng loại trực tiếp.
5. **Chờ BTC duyệt:** xem replay, nhật ký và kết quả, sau đó bấm **Công bố kết quả**.

Tạm dừng chấm mới ngăn cấp công việc mới; trận đang chạy được phép hoàn tất. Bấm lại để tiếp tục. Lỗi hạ tầng được thử tối đa 3 lần, sau đó hiện ở hàng đợi lỗi để BTC sửa worker và bấm thử lại. Không chấm lại kết quả đã công bố.

4. Sinh viên: viết và nộp bot

Đăng nhập, mở **Bài nộp của đội**, nhập mã BTC gửi. Chọn đúng giải; tên đội hiện cạnh ô nộp bài. Có thể dán code hoặc đọc một tệp `.py` UTF-8 tối đa 64 KiB.

```
def choose_move(state):
    moves = state["legal_moves"]
    return {"from": moves[0]["from"],
            "to": moves[0]["to"]}
```

`state` có `board`, `player`, `legal_moves`, `turn`, `history`. `board[y][x]` bắt đầu ở A1; mỗi ô là `None` hoặc đối tượng có `player` và `type` (ROCK/PAPER/SCISSORS). `player` là P1/P2. `turn` là số nước đã đi, bắt đầu từ 0. Chọn từ `legal_moves` để tránh nước không hợp lệ.

Bot được giữ trạng thái trong một ván, nhưng khởi tạo lại ở ván tiếp theo. Không in debug ra stdout; có thể dùng stderr với lượng nhỏ. Không dùng mạng, subprocess, pip hay extension native. Python WASI chỉ hỗ trợ một tập con thư viện chuẩn; kết quả kiểm tra chính thức mới quyết định tương thích.

Ba mẫu tải được tại `/bots/sprinter.py`, `/bots/guardian.py`, `/bots/tactician.py`. Có thể luyện tập ở giao diện cũ nhưng điểm luyện tập không được tính vào giải.

Sau khi bấm nộp

Hệ thống lưu source, SHA-256 và giờ nhận trên máy chủ, rồi kiểm tra bot ở cả hai phía với bàn cờ đã thay đổi. Trạng thái là chờ kiểm tra, hợp lệ hoặc chưa hợp lệ; lỗi hiện ngay dưới phiên bản. Hợp lệ chỉ là đạt kiểm tra cơ bản; bot vẫn có thể thua vì lỗi khi gặp trạng thái khác trong trận.

Mỗi đội có tối đa 2 bài chờ, 1 phiên bản/5 giây và 200 phiên bản/giải. Nộp lại cùng source trả về cùng bài. Bài hợp lệ mới nhất được nhận trước hạn là bài được chọn, không nhất thiết là bài vừa nộp cuối cùng nếu bài đó lỗi.

5. Git, thể thức và xem kết quả

Nộp bằng Git

BTC có thể gắn `owner/repo` công khai cho đội. Trong mục nộp Git, sinh viên nhập commit SHA đầy đủ 40 ký tự. Máy chủ chỉ lấy tệp `bot.py` ở thư mục gốc từ repo đã gắn, không clone, chạy hook hoặc cài dependency.

Danh tính vẫn lấy từ tài khoản và mã đội, không lấy tên tác giả Git. Thời điểm nộp là khi máy chủ nhận xong source, không phải ngày commit. Repo riêng tư dùng tải `.py` lên; phiên bản này chưa có GitHub App/OAuth hoặc webhook.

Luật và điểm

Bàn cờ 9 × 9, xuất phát FRONTLINE. P1 tới I1, P2 tới A9; mỗi quân đi tối đa một ô theo 8 hướng. Ăn quân theo ROCK/PAPER/SCISSORS; cùng loại chặn nhau. Không còn nước đi hợp lệ thì thua. Bot lỗi, đi sai, vượt thời gian hoặc giới hạn output có thể bị xử thua. Khi cả hai lỗi lúc khởi tạo, P1 được kiểm tra trước; đổi bên ở ván sau cân bằng thứ tự này.

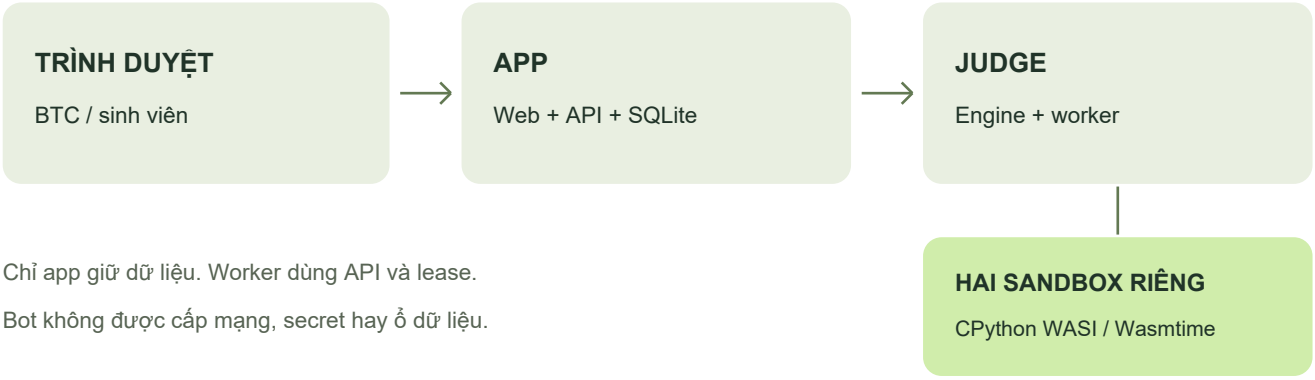
Mỗi cặp chơi 2 ván đổi bên. Thắng ván được 1, hòa được 0,5; cộng hai ván để chốt cặp thắng/hòa/thua và nhận 3/1/0 điểm vòng bảng. Bằng điểm: đối đầu trong toàn nhóm bằng điểm, hiệu số ván, rồi hạt giống nhỏ hơn.

Các đội đi tiếp tạo nhánh loại trực tiếp tự động. Cặp hòa chơi thêm tối đa `extraSets` cặp đổi bên; vẫn hòa thì hạt giống nhỏ hơn đi tiếp. Quy tắc phải được công khai trước giải.

Tra cứu

Giải đấu: bảng điểm và nhánh loại trực tiếp. **Lịch đấu & replay:** 50 ván gần đây, phát từng nước hoặc tải JSON; tối đa 500 cặp lịch hiển thị. Nút CSV tải bảng điểm. BTC có bản xuất JSON toàn giải gồm cấu hình, entries, source được chọn, kết quả và audit; không chia sẻ công khai bản xuất có source nếu chưa được phép.

6. Mỗi máy chạy gì?



Chỉ app giữ dữ liệu. Worker dùng API và lease.

Bot không được cấp mạng, secret hay ổ dữ liệu.

Một máy Linux hoặc Docker Desktop có thể chạy cả hai container khi thử nghiệm. Khi mở rộng, giữ app và ổ dữ liệu trên máy điều hành; đưa worker sang máy Linux riêng. Worker gọi API qua HTTPS, không mount ổ database và không mở cổng nhận kết nối từ sinh viên.

Máy / tiến trình	Phần việc
Máy điều hành: app	Next.js production, đăng nhập, API, chia bảng, hàng đợi SQLite, kiểm tra replay và chốt kết quả
Ổ dữ liệu local	auth.sqlite: tài khoản/session; competition.sqlite: giải, bài, job, replay; casual.sqlite: luyện tập/phòng bạn bè
Máy chấm: judge	Một slot kiểm tra bài và một slot chấm trận, lấy job có lease, heartbeat 20 giây
Sandbox bot	Tiến trình Wasmtime riêng; CPython 3.15.0 trong WebAssembly, chỉ đọc thư viện Python

Luồng xử lý

Trình duyệt → session và membership → source/hash cố định → hàng đợi bền vững → worker có lease → hai sandbox → luật cờ đáng tin cậy → replay → transaction ghi kết quả một lần.

Bot không được thấy secret worker, database, source đối thủ hay filesystem máy chủ. WebAssembly chặn đường gọi hệ điều hành; Wasmtime giới hạn bộ nhớ/fuel; supervisor giới hạn thời gian và output; Docker giới hạn tổng CPU, RAM và PID. Không dùng AST làm ranh giới an toàn.

Lease có hạn 90 giây và được gia hạn. Nếu máy chấm chết, lease mới thay lease cũ; kết quả từ worker cũ bị từ chối. Replay phải đúng phiên bản engine/runtime, source hash và seed. Ghi kết quả, chốt cặp và tạo vòng sau trong cùng transaction, nên gửi lặp không cộng điểm hai lần.

Nếu cập nhật phần mềm làm phiên bản engine/runtime khác job đã khóa, giải tự tạm dừng và ghi nhật ký. Khôi phục image tương thích trước khi tiếp tục; hệ thống không âm thầm dùng luật mới để chấm bài cũ.

7. Sao lưu và phục hồi

Làm trước giải và trước khi cập nhật hệ thống. Dừng cả hai dịch vụ để bản chụp ba database nhất quán; không sao chép tệp SQLite đang ghi cùng WAL một cách thủ công.

```
docker compose stop app judge
docker compose run --rm --no-deps app node scripts/backup-competition.mjs /data/backups/b1
mkdir backups
docker compose cp app:/data/backups/b1 ./backups/b1
docker compose up -d
```

Mỗi lần dùng tên thư mục mới, ví dụ b2. Tiện ích dùng SQLite backup API và chạy integrity_check. Giữ một bản ngoài máy chủ; sao lưu `.secrets/judge-token` riêng bằng kênh bảo mật. Tệp backup chứa dữ liệu tài khoản và source, không đưa lên repo.

Thử phục hồi sang volume mới

Giữ nguyên volume đang dùng. Tạo một volume rỗng, nạp bản sao đã kiểm tra:

```
docker volume create arena-restored
docker run --rm --user 0 `
  -v arena-restored:/data `
  -v "${PWD}/backups/b1:/restore:ro" `
  oan-tu-ti-arena:local node scripts/restore-empty.mjs
```

Tiện ích từ chối nếu volume đích không rỗng. Sau khi dừng app/judge, đặt dòng sau vào tệp `.env` ở gốc dự án và khởi động lại:

```
ARENA_DATA_VOLUME=arena-restored
```

Kiểm tra đăng nhập, bài nộp, điểm và replay. Muốn quay lại bản gốc, dừng dịch vụ, bỏ dòng override rồi khởi động lại. Volume gốc được giữ nguyên. Kết quả/job đang chạy trong bản backup sẽ được lấy lại sau khi lease hết hạn.

8. Đưa lên Internet và mở rộng

Gói deploy kèm Caddy để cấp HTTPS. Cần máy Linux có IP/domain, Docker, DNS trỏ đúng và cổng 80/443. Tạo `.env`:

```
ARENA_DOMAIN=arena.example.edu
```

Sau đó chạy từ gốc repo:

```
npm run competition:init
docker compose -f compose.yaml -f deploy/compose.public.yaml up -d --build --wait
```

Thay domain ví dụ bằng domain thật. Trình duyệt chỉ đi qua HTTPS, `PUBLIC_ORIGIN` khớp domain, cookie có Secure; chỉ app được giữ database. Không đưa cổng 8787, Docker socket hoặc database ra ngoài. Secret giữa app và worker cần được luân chuyển cùng lúc nếu lộ.

Có thể thêm worker trên cùng máy khi đã đo đủ RAM/CPU:

```
docker compose up -d --scale judge=2
```

Mỗi replica mặc định có một slot trận và một slot validation, giới hạn container 2 GB RAM và 2 CPU. Tăng replica không làm CPU vật lý tự tăng. Máy chấm riêng dùng cùng image, `CONTROL_URL` HTTPS và tệp token được cấp bảo mật.

Quy mô và chi phí

Một app SQLite trên SSD local giảm chi phí và số dịch vụ phải quản trị. Không nhân bản app cùng ghi một file qua NFS. Khi cần nhiều web server, chuyển store và queue sang PostgreSQL có transaction tương ứng.

16 đội, 4 bảng, lấy 2 đội/bảng: 48 ván vòng bảng + 14 ván loại trực tiếp, chưa tính cặp phụ. Với 4.000 bot và 128 bảng, vòng bảng có 121.024 ván; riêng số trận đã lớn. Thời gian gần đúng = tổng ván × thời gian trung bình/ván ÷ số slot, cộng validation và hiệp phụ. Đo bot thật trước khi mua thêm máy. Chưa có chứng nhận chịu tải 4.000 người đồng thời.

9. Kiểm tra và xử lý sự cố

```
npm test
npm run test:competition
npx tsc --noEmit
docker compose run --rm `
  -v "${PWD}/tests:/app/tests:ro" `
  -v "${PWD}/public/bots:/app/public/bots:ro" `
  judge node tests/judge.test.mjs
npm run test:docker
npm run test:competition:e2e
```

E2E chỉ chạy trên localhost, tạo tài khoản/giải DEMO và cấp quyền BTC thử bằng Docker CLI. Không chạy công cụ tạo dữ liệu thử vào một giải đang tổ chức. Workflow GitHub chạy cùng các kiểm tra này trên môi trường tách biệt.

Hiện tượng	Cách xử lý
Không có menu BTC	Đăng ký trước, cấp quyền đúng email bằng CLI, tải lại trang
Mã tham gia sai	Kiểm tra email tài khoản, hạn 7 ngày, mã đã dùng; BTC cấp lại riêng
Bài chờ lâu	Xem trạng thái worker/hàng đợi; kiểm tra Docker logs judge, token và phiên bản runtime
Không khóa được	Chưa tới hạn, còn bài pending, số đội không đủ hoặc quá 32 đội/bảng
Job lỗi sau 3 lần	Sửa worker/network/tài nguyên, bấm thử lại; không tự đổi kết quả
Lỗi 403 khi nộp	Đúng membership, Origin/domain/port, session còn hạn; không giả header danh tính
Bot chạy practice nhưng lỗi chính thức	Kiểm tra thư viện WASI, hạn bộ nhớ/fuel, thời gian và stdout
HTTP 503 trên Vercel	Bản luyện tập không có backend giải chính thức; mở địa chỉ Docker do BTC cấp

Phạm vi đã kiểm chứng

Đã chạy kiểm thử engine, tài khoản/CSRF/phân quyền/phòng bạn bè; xử lý lease hết hạn, kết quả gửi lặp, replay giả, đóng băng config và chốt giải; bot Python thật cùng mẫu truy cập tệp, mạng, subprocess, tràn output/bộ nhớ và vòng lặp vô hạn. Một giải 16 đội/62 ván đã hoàn tất bằng worker thật.

Trước giải lớn: thử tải đăng nhập/nộp bài trên mạng trường, đo thời gian bot thật, diễn tập mất worker và restore backup. Phiên bản này chưa có SSO trường, email verification/recovery tự động, streaming export giải rất lớn hoặc kiểm định an toàn độc lập. Không coi kết quả kiểm thử chức năng là bảo đảm không có lỗ hổng.